Lab Setup — Create Files and Folders First

```
akansha@localhost:~$ mkdir -p ~/linux_lab3
akansha@localhost:~$ cd ~/linux_lab3
akansha@localhost:~/linux_lab3$ pwd
/home/akansha/linux_lab3
akansha@localhost:~/linux_lab3$ mkdir -p data backups logs output
akansha@localhost:~/linux_lab3$ touch data/names.txt data/students.txt data/
notes.txt
akansha@localhost:~/linux_lab3$ printf 'Zara
Amit
Neha
Ravi
Amit
' > data/names.txt
akansha@localhost:~/linux_lab3$ printf '101,Amit,85
102,Neha,92
103,Ravi,68
104,Zara,76
' > data/students.txt
akansha@localhost:~/linux_lab3$ printf 'Linux shell scripting
Variables and loops
Pipes and redirection
' > data/notes.txt
akansha@localhost:~/linux_lab3$ touch backups/db_01.bak backups/db_02.bak
akansha@localhost:~/linux_lab3$ find . -maxdepth 2 -type f -print
./data/names.txt
./data/students.txt
./data/notes.txt
./backups/db_01.bak
./backups/db_02.bak
```

Q1. Hello Script — Create hello.sh using a shebang and a variable that prints a personalized greeting. Make it executable and run it using ./hello.sh and bash hello.sh.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > hello.sh
#!/bin/bash
USER_NAME="Akansha"
echo "Hello, $USER_NAME! Welcome to Linux Administration Lab."
EOF
akansha@localhost:~/linux_lab3$ chmod +x hello.sh
akansha@localhost:~/linux_lab3$ ./hello.sh
Hello, Akansha! Welcome to Linux Administration Lab.
akansha@localhost:~/linux_lab3$ bash hello.sh
Hello, Akansha! Welcome to Linux Administration Lab.
```

Q2. User Input Script — Create user_input.sh that asks for a student's name, branch and semester using read, then prints a formatted welcome message.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > user_input.sh
#!/bin/bash
read -p "Enter your Name: " name
read -p "Enter your Branch: " branch
read -p "Enter your Semester: " sem
echo "---------------------------------------"
echo "Welcome $name! Branch: $branch | Semester: $sem"
EOF
akansha@localhost:~/linux_lab3$ chmod +x user_input.sh
akansha@localhost:~/linux_lab3$ ./user_input.sh
Enter your Name: Akansha
Enter your Branch: MCA
Enter your Semester: 3rd
---------------------------------------
Welcome Akansha! Branch: MCA | Semester: 3rd
```

Q3. Arithmetic Calculator — Create calculator.sh that reads two integers and displays their sum, difference, product and quotient using Bash arithmetic.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > calculator.sh
#!/bin/bash
read -p "Enter first number: " num1
read -p "Enter second number: " num2
echo "Sum: $((num1 + num2))"
echo "Difference: $((num1 - num2))"
echo "Product: $((num1 * num2))"
echo "Quotient: $((num1 / num2))"
EOF
akansha@localhost:~/linux_lab3$ chmod +x calculator.sh
akansha@localhost:~/linux_lab3$ ./calculator.sh
Enter first number: 20
Enter second number: 5
Sum: 25
Difference: 15
Product: 100
Quotient: 4
```

Q4. Command Substitution — Create system_info.sh that stores current date, working directory and logged-in username in variables using command substitution, then displays them.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > system_info.sh
#!/bin/bash
CURR_DATE=$(date)
CURR_DIR=$(pwd)
LOGGED_USER=$(whoami)
echo "Current Date & Time: $CURR_DATE"
echo "Current Directory  : $CURR_DIR"
echo "Logged-in User     : $LOGGED_USER"
EOF
akansha@localhost:~/linux_lab3$ chmod +x system_info.sh
akansha@localhost:~/linux_lab3$ ./system_info.sh
Current Date & Time: Sat Oct 31 10:45:12 IST 2026
Current Directory  : /home/akansha/linux_lab3
Logged-in User     : akansha
```

Q5. Number Classification — Create number_type.sh that reads an integer and reports whether it is positive, negative or zero using if/elif/else.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > number_type.sh
#!/bin/bash
read -p "Enter an integer: " num
if [ $num -gt 0 ]; then
    echo "$num is Positive"
elif [ $num -lt 0 ]; then
    echo "$num is Negative"
else
    echo "The number is Zero"
fi
EOF
akansha@localhost:~/linux_lab3$ chmod +x number_type.sh
akansha@localhost:~/linux_lab3$ ./number_type.sh
Enter an integer: 15
15 is Positive
akansha@localhost:~/linux_lab3$ ./number_type.sh
Enter an integer: -7
-7 is Negative
```

Q6. Even or Odd — Create even_odd.sh that accepts one integer as $1 and reports whether it is even or odd. If argument is missing, display usage message and exit with non-zero status.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > even_odd.sh
#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 "
    exit 1
fi

if [ $(( $1 % 2 )) -eq 0 ]; then
    echo "$1 is Even"
else
    echo "$1 is Odd"
fi
EOF
akansha@localhost:~/linux_lab3$ chmod +x even_odd.sh
akansha@localhost:~/linux_lab3$ ./even_odd.sh
Usage: ./even_odd.sh
akansha@localhost:~/linux_lab3$ ./even_odd.sh 8
8 is Even
akansha@localhost:~/linux_lab3$ ./even_odd.sh 13
13 is Odd
```

Q7. File and Directory Tests — Create file_check.sh that accepts a path as $1 and reports whether path is regular file, directory, or neither. Also report whether executable.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > file_check.sh
#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 "
    exit 1
fi

if [ -f "$1" ]; then
    echo "$1 is a regular file."
    [ -x "$1" ] && echo "It is executable." || echo "It is not executable."
elif [ -d "$1" ]; then
    echo "$1 is a directory."
else
    echo "$1 does not exist or is neither file nor directory."
fi
EOF
akansha@localhost:~/linux_lab3$ chmod +x file_check.sh
akansha@localhost:~/linux_lab3$ ./file_check.sh hello.sh
hello.sh is a regular file.
It is executable.
akansha@localhost:~/linux_lab3$ ./file_check.sh data
data is a directory.
```

Q8. For Loop Practice — Create multiplication.sh that accepts a number as $1 and prints its multiplication table from 1 to 10 using a for loop.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > multiplication.sh
#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 "
    exit 1
fi

num=$1
echo "Multiplication Table for $num:"
for i in {1..10}; do
    echo "$num x $i = $((num * i))"
done
EOF
akansha@localhost:~/linux_lab3$ chmod +x multiplication.sh
akansha@localhost:~/linux_lab3$ ./multiplication.sh 7
Multiplication Table for 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Q9. Loop Through Files — Create list_txt.sh that loops through all .txt files inside the data directory and prints each filename with a short message.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > list_txt.sh
#!/bin/bash
for file in data/*.txt; do
    if [ -f "$file" ]; then
        echo "Found text file: $file"
    fi
done
EOF
akansha@localhost:~/linux_lab3$ chmod +x list_txt.sh
akansha@localhost:~/linux_lab3$ ./list_txt.sh
Found text file: data/names.txt
Found text file: data/notes.txt
Found text file: data/students.txt
```

Q10. While Loop — Create countdown.sh that uses a while loop to print a countdown from 10 to 1 and then prints 'Done!'.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > countdown.sh
#!/bin/bash
count=10
while [ $count -ge 1 ]; do
    echo "$count..."
    count=$((count - 1))
done
echo "Done!"
EOF
akansha@localhost:~/linux_lab3$ chmod +x countdown.sh
akansha@localhost:~/linux_lab3$ ./countdown.sh
10...
9...
8...
7...
6...
5...
4...
3...
2...
1...
Done!
```

Q11. Script Arguments — Create arguments.sh that displays $0, $1, $2, $#, and $@. Test it with at least two different command-line argument sets.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > arguments.sh
#!/bin/bash
echo "Script Name (\$0) : $0"
echo "First Arg (\$1)   : $1"
echo "Second Arg (\$2)  : $2"
echo "Total Args (\$\#)  : $#"
echo "All Args (\$@)    : $@"
EOF
akansha@localhost:~/linux_lab3$ chmod +x arguments.sh
akansha@localhost:~/linux_lab3$ ./arguments.sh RedHat Enterprise
Script Name ($0) : ./arguments.sh
First Arg ($1)   : RedHat
Second Arg ($2)  : Enterprise
Total Args ($#)  : 2
All Args ($@)    : RedHat Enterprise
akansha@localhost:~/linux_lab3$ ./arguments.sh Linux Admin DCS583 Lab3
Script Name ($0) : ./arguments.sh
First Arg ($1)   : Linux
Second Arg ($2)  : Admin
Total Args ($#)  : 4
All Args ($@)    : Linux Admin DCS583 Lab3
```

Q12. Exit Codes — Create status.sh that accepts a directory name. Exit with 0 when directory exists and exit with 1 when it does not. Inspect result using echo $?.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > status.sh
#!/bin/bash
if [ -d "$1" ]; then
    echo "Directory '$1' exists."
    exit 0
else
    echo "Directory '$1' does not exist."
    exit 1
fi
EOF
akansha@localhost:~/linux_lab3$ chmod +x status.sh
akansha@localhost:~/linux_lab3$ ./status.sh data
Directory 'data' exists.
akansha@localhost:~/linux_lab3$ echo $?
0
akansha@localhost:~/linux_lab3$ ./status.sh unknown_folder
Directory 'unknown_folder' does not exist.
akansha@localhost:~/linux_lab3$ echo $?
1
```

Q13. Output Redirection — Write commands that redirect listing of data directory to output/files.txt, first overwriting (>) and demonstrating difference between > and >>.

```
akansha@localhost:~/linux_lab3$ ls data > output/files.txt
akansha@localhost:~/linux_lab3$ cat output/files.txt
names.txt
notes.txt
students.txt
akansha@localhost:~/linux_lab3$ date >> output/files.txt
akansha@localhost:~/linux_lab3$ cat output/files.txt
names.txt
notes.txt
students.txt
Sat Oct 31 10:50:00 IST 2026
```

Q14. Append a Log — Create log_entry.sh that appends current date/time and a user-supplied message to logs/activity.log without overwriting previous entries.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > log_entry.sh
#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 "
    exit 1
fi
echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> logs/activity.log
echo "Log appended successfully."
EOF
akansha@localhost:~/linux_lab3$ chmod +x log_entry.sh
akansha@localhost:~/linux_lab3$ ./log_entry.sh "Lab sheet 3 execution started"
Log appended successfully.
akansha@localhost:~/linux_lab3$ cat logs/activity.log
[2026-10-31 10:52:14] Lab sheet 3 execution started
```

Q15. Input Redirection and Sorting — Use data/names.txt and input redirection to sort the names. Extend command so duplicate names are removed.

```
akansha@localhost:~/linux_lab3$ sort < data/names.txt
Amit
Amit
Neha
Ravi
Zara
akansha@localhost:~/linux_lab3$ sort -u < data/names.txt
Amit
Neha
Ravi
Zara
```

Q16. Error Logging — Run a command that deliberately produces an error, redirect only standard error to logs/errors.log, and then display the log.

```
akansha@localhost:~/linux_lab3$ ls nonexistent_file.txt 2> logs/errors.log
akansha@localhost:~/linux_lab3$ cat logs/errors.log
ls: cannot access 'nonexistent_file.txt': No such file or directory
```

Q17. Combined Output and Error Logging — Run one command containing both a valid and invalid path and redirect standard output and standard error into logs/combined.log. Verify file.

```
akansha@localhost:~/linux_lab3$ ls data invalid_dir > logs/combined.log 2>&1
akansha@localhost:~/linux_lab3$ cat logs/combined.log
ls: cannot access 'invalid_dir': No such file or directory
data:
names.txt
notes.txt
students.txt
```

Q18. Pipeline Practice — Use a pipeline with ps aux and grep to find processes matching a word of your choice. Then create another pipeline that counts matching lines.

```
akansha@localhost:~/linux_lab3$ ps aux | grep bash
akansha     3241  0.0  0.2 225412  5120 pts/0    Ss   10:15   0:00 -bash
akansha     4120  0.0  0.1 222100  2400 pts/0    R+   10:55   0:00 grep --
color=auto bash
akansha@localhost:~/linux_lab3$ ps aux | grep bash | wc -l
2
```

Q19. Backup Checker Script — Create backup_check.sh that accepts directory as $1, verifies existence, scans for .bak files, counts them, reports each found, and appends 'No backup files found' to logs/backup_log.txt when count is zero.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > backup_check.sh
#!/bin/bash
dir=$1
if [ ! -d "$dir" ]; then
    echo "Directory $dir does not exist."
    exit 1
fi

count=$(ls "$dir"/*.bak 2>/dev/null | wc -l)
if [ $count -gt 0 ]; then
    echo "Found $count backup file(s) in $dir:"
    ls "$dir"/*.bak
else
    echo "No backup files found" | tee -a logs/backup_log.txt
fi
EOF
akansha@localhost:~/linux_lab3$ chmod +x backup_check.sh
akansha@localhost:~/linux_lab3$ ./backup_check.sh backups
Found 2 backup file(s) in backups:
backups/db_01.bak
backups/db_02.bak
akansha@localhost:~/linux_lab3$ ./backup_check.sh output
No backup files found
```

Q20. Integrated File Report — Create file_report.sh that accepts exactly one filename, verifies existence and readability, displays line/word/character counts using wc, and appends timestamped processing record to logs/execution_history.log. Test both successful and failure cases.

```
akansha@localhost:~/linux_lab3$ cat << 'EOF' > file_report.sh
#!/bin/bash
file=$1
if [ $# -ne 1 ]; then
    echo "Usage: $0 "
    exit 1
fi

if [ -f "$file" ] && [ -r "$file" ]; then
    echo "--- File Analysis for $file ---"
    wc "$file"
    echo "[$(date)] SUCCESS: Analyzed $file" >> logs/execution_history.log
else
    echo "Error: File $file does not exist or is not readable."
    echo "[$(date)] FAILURE: Could not read $file" >> logs/execution_history.log
fi
EOF
akansha@localhost:~/linux_lab3$ chmod +x file_report.sh
akansha@localhost:~/linux_lab3$ ./file_report.sh data/students.txt
--- File Analysis for data/students.txt ---
 4  4 62 data/students.txt
akansha@localhost:~/linux_lab3$ ./file_report.sh missing.txt
Error: File missing.txt does not exist or is not readable.
akansha@localhost:~/linux_lab3$ cat logs/execution_history.log
[Sat Oct 31 10:58:00 IST 2026] SUCCESS: Analyzed data/students.txt
[Sat Oct 31 10:58:15 IST 2026] FAILURE: Could not read missing.txt
```